

안리안

Frontend Developer

2000년 (만 26세)

Email 00rian@naver.com | GitHub | 기술 블로그

자기소개

문제를 눈으로 보이는 지표로 바꾸는 프론트엔드 개발자 입니다.
다양한 의견을 나누며 **서로 검증할 수 있는 협업 환경**을 좋아합니다.
기억은 흐려져도 **기록은 남는다**고 생각해 **배운걸 글로 정리하며 생각을 완성**합니다.
기획부터 배포, 그로스까지 전체 사이클을 책임지며 **프로덕션 장애를 직접 진단, 해결**한 경험이 있습니다.

프로젝트

마패 (MaskPass)

얼굴 인식 기반 입장관리 시스템
2025.02 - 2025.04

팀 구성 : PM/PD 2명, FE 3명, BE 4명

기술 스택
React, styled-components, TypeScript, Vite, react-webcam, face-api, tanstack-query

블로그 링크

성능 최적화

1. **Face-api 감지 루프 최적화로 불필요한 네트워크 통신 및 메인 스레드 점유 감소**

   [문제 상황]
   얼굴 인식 범위 안에 들어올 때마다 스크린샷을 캡처 및 백엔드로 전송하는 과정에서 해당 통신이 매 프레임마다 지속적으로 발생

   [원인 분석]
   Chrome DevTools Performance 탭 녹화 시 Memory 항목을 함께 활성화해 확인한 결과, Scripting 구간이 길게 이어지고(base64디코딩 후 배열을 두 번 순회하는 비효율적인 구현) JS Heap 크기가 캡처 요청이 몰릴 때마다 계단식으로 상승했다가 GC 시점에 떨어지는 톱니 패턴이 반복되는 것을 확인
   -> 응답을 기다리는 동안 캡처 된 이미지 payload가 동시다발적으로 누적되고 있었음이 원인

   [해결]
   face-api의 특징점 유사도 비교로 이미 인증된 얼굴인지 먼저 판별하고, 인증 진행 중엔 감지 루프를 일시 중단하는 구조를 도입해 불필요한 캡처, 통신 자체를 제거.

2. **SSE 연결 메모리 누수 원인 특정 및 3단계 수정으로 리소스 정리 구조 안정화**

[문제 상황]

관리자 대시보드 실시간 입장 카운팅에 사용하는 SSE 훅에서 페이지 이동 시마다 연결이 정상 종료되지 않고 누적되는 문제 발생

[원인 분석]

Chrome DevTools Network 탭에서 EventStream 요청을 필터링해 확인한 결과, 관리자 페이지를 반복해서 드나들 때마다 **SSE 연결이 종료되지 않고 pending 상태로 계속 쌓이는 것을 확인.**

이를 바탕으로 cleanup 함수를 코드 레벨에서 추적한 결과, 기존에 열려 있던 **EventSource 인스턴스를 닫는 게 아니라 동일한 URL로 새 EventSource를 생성해 그 새 연결만 닫고 있던 것이 원인**

[해결]

연결 생성 시점의 실제 인스턴스를 배열에 담아 정확히 그 연결을 닫도록 수정. 1차 수정 과정에서 useEffect 의존성 배열에 매 렌더링마다 참조가 바뀌는 배열을 그대로 추가해 무한 재구독 루프가 발생하는 것을 확인, 배열을 의존성에서 제거하는 대신 세션 ID 목록을 문자열로 변환(sessionIds.join(','))해 원시 값 기준으로 비교되도록 의존성 구조를 재설계

최종적으로 클라이언트 측 연결 종료(close())와 서버 측 리소스 해제(unsubscribe API 호출)를 **항상 쌍으로** 수행하도록 구조화 하며 **불필요한 재렌더링으로 인한 재연결 없이, 세션 구성이 실제로 바뀔 때만 연결을 새로 맺도록 개선해 연결 누적 없이 안정적으로 리소스를 유지**

코드 설계

1. **확장성, 재사용성을 고려한 아키텍쳐 설계**

Custom Hook을 이용한 관심사 분리 및 재사용성

얼굴 등록/인증이 인식->캡처->통신이라는 동일 흐름을 공유하는 상황에서 공통 로직(감지, 캡처)은 훅에 고정하고 후처리는 콜백으로 위임하는 구조를 적용

(콜백 내용: 등록은 단순 캡처, 인증은 특징점 거리 비교 기반)

이후 새로운 얼굴 인식을 활용하는 기능이 추가되더라도 훅 내부 구조 변경 없이 콜백만 새로 정의해 넘겨주면 되도록 확장성 확보

이외에도 실시간 입장 카운팅(useSSE), 사용자·컨퍼런스 서버 상태 조회(useUserData, useConferenceData) 등 도메인 로직을 각각 훅으로 분리

컴포넌트는 UI 렌더링에만 집중하고, 데이터 fetch, 구독, 해제 같은 부수 효과는 내부에 캡슐화

2. **Zustand로 전역 인증 상태 관리(props drilling 방지)**

[상황]

로그인 사용자 정보를 라우트 가드, 대시보드, 사용자 카드 등 여러 레이어에서 공통으로 참조

[방식]

단일 스토어 (useAuthStore)로 분리해 컴포넌트 **트리를 거치지 않고 직접 접근하도록 설계**

(라우트 가드 2곳, 페이지 3곳, 컴포넌트 2곳, 훅 1곳에서 사용)

3. 불필요한 상태 제거를 이용한 재렌더링, 데이터 불일치 방지

얼굴 삭제 후 사용자 정보 갱신 로직이 **로컬 state(useState)로 서버 데이터를 복제**해 관리하고 있어, 해당 컴포넌트만 최신화되고 Header 등 다른 화면은 이전 상태를 참조하는 **불일치가 발생할 수 있는 구조** -> 로컬 state를 제거하고 React Query 훅의 refetch 호출로 대체, 쿼리 refetch 시 **전역 스토어 (Zustand)까지 함께 동기화**되도록 구조화해 상태를 단일화.

이와 함께 사용자, 컨퍼런스 정보 조회를 queryKey 기반 캐싱 계층으로 통합해, 동일 데이터를 참조하는 여러 컴포넌트가 각자 **API를 재호출하지 않고 캐시를 공유**하도록 설계

4. 공통 로직의 선언적 캡슐화

인증/인가 검사를 각 페이지 컴포넌트마다 반복 작성하는 대신, **PrivateRoute/PublicRoute 가드 컴포넌트로 분리해 라우터 레이어에서 선언적으로 처리**

미인증시 초기화면으로, 역할(role) 불일치 시 각자의 대시보드로 **조건부 리다이렉트**하는 로직을 **페이지 코드에서 완전히 제거**

5. 연산 겹침 및 낭비 방지 감지 루프 설계 (setInterval, requestAnimationFrame)

setInterval은 고정 주기로 무조건 콜백을 실행하는 방식이라, face-api 추론 시간이 주기보다 길어지면 이전 추론이 끝나기 전에 다음 추론이 시작되는 겹침(overlap) 문제가 생길 수 있고 탭이 비활성화되어도 계속 실행돼 불필요한 리소스를 소모한다고 판단

requestAnimationFrame의 "이전 연산이 끝난 뒤에만 다음 호출을 스스로 예약"하는 재귀 구조와 "탭 비활성화 시 브라우저가 자동으로 호출을 멈추는" 특성이 이 두 단점을 보완할 수 있다고 판단.

이를 활용해 **별도의 상태 플래그 없이**도 연산이 겹치지 않고 **화면을 보고 있지 않을 때는 자동으로 멈추는 얼굴 감지 루프를 구현**

6. CSS - 디자인 토큰 구조 설계

원시 색상 값을 CSS 커스텀 프로퍼티로 정의하고, background/ border/divider/icon/typo 카테고리별 **시맨틱 토큰(lightTheme.ts) 으로 매핑 -> 컴포넌트에서 직접 색상 값 대신 의미 단위(ex: icon.error)**로 참조하도록 설계, 반응형 브레이크포인트도 별도 토큰으로 관리해 **디자인 변경 시 컴포넌트 수정 없이 토큰 교체만으로 전체 반영**되는 구조 확보)

협업/UT

1. **의존성 분석으로 작업 우선순위 결정**
   중간 점검 시점까지 기획·디자인 시스템·백엔드 API가 모두 확정되지 않은 상황에서, **외부 의존성 유무를 기준으로 우선순위 리스트를 구성** - 얼굴 인식, 얼굴 등록, 컨퍼런스 리스트업 등 **핵심 기능을 최우선 착수 대상**으로, 마이페이지, 예약내역 확인 등 부가 기능을 후순위로 배치.

   이후 디자인 시스템(공통 컴포넌트)이나 API가 하나씩 확정될 때마다 **UI를 제외한 기능적 로직부터 먼저 구현**해두는 방식으로 순서를 조율해 팀 **전체가 대기 시간 없이 작업**할 수 있도록 함
2. **UT 기반 검증으로 기존 수기 방식 대비 평균 출입 시간 약 1분 56초 단축**
   신규 방식(얼굴 인식 입장)이 기존 방식(수기 서명 입장) 대비 얼마나 빨라지는지 **시간으로 측정**하기 위해, 참가자 전원에게 입장 절차를 동일하게 사전 안내한 뒤 진행하는 방식으로 UT 설계.

   25.03.29 오프라인 UT 진행(A/B 2타임, 총 16명). **실제 현장에서 발생할 수 있는 예외 상황까지 시나리오**에 포함(사전 미예매자는 수기 방식으로, 사전 미등록자는 현장 얼굴 등록 후 인식하는 흐름까지 검증)

   - 시간대별 소요 시간(A: 5분 07초 -> 2분 21초, B: 3분 05초->1분 58초) 측정 결과 **평균 1분 56초 단축**, 인터뷰에서 **응답자의 100%가 속도면에서**, 87.5%가 대규모 행사 적합성 면에서 마스크패스 우수하다고 응답

스무디

SNS 기반 유행 템플릿 제작 플랫폼
2026.03 - 2026.09

팀 구성 : FE 1명, BE 1명

기술 스택
Next.js, TypeScript, Tailwind CSS, TanStack Query, axios

깃헙 링크

- **테스트 인프라 부재 프로젝트에 Playwright E2E 체계 처음부터 설계·구축 (68개 테스트)** 해당 PR
  기존 테스트 코드가 전무했던 프로젝트에서 컬러워크·마니또 월드 2개 핵심 기능의 PRD와 실제 구현 코드를 대조해 **스펙 문서화** → **프론트엔드 단독으로** 구동되는 테스트 아키텍처로 **68개 시나리오 설계·구현** → 그 과정에서 인가 검증 누락(비로그인 요청으로 타인 리소스 접근 가능), 클릭 가로채기, 도달 불가능한 죽은 UI 등 프로덕션 버그 3건 발견
- **Vitest 기반 유닛테스트 도입 및 CI 파이프라인 통합** 해당 PR
  순수 함수(색상 코드→이름 변환, 팔레트 랜덤 추출)에 **정상/엣지 케이스 유닛테스트 작성**, jsdom 기반 테스트 환경 구성 → GitHub Actions CI의 프론트엔드 job에 유닛테스트 실행 단계를 추가해 PR마다 자동 검증되도록 파이프라인 통합
- **Next.js standalone 빌드 기반 멀티스테이지 Dockerfile 작성으로 배포 이미지 최적화** 해당 PR
  output: standalone 모드와 멀티스테이지 빌드로 **실행에 필요한 파일만 포함**하도록 이미지 구성.
  동일 프로젝트를 일반 방식(node_modules 전체 포함)으로 빌드한 이미지(1.72GB)와 비교했을 때 **383MB로 약 78% 용량 감소**

손사주

사주와 손금을 결합한 AI운세 서비스 (출시 후 사용자 약 200명)
2026.09

개인프로젝트

기술 스택
- **FE:** Next.js, TypeScript, Tailwind CSS
- **Test:** Vitest, Playwright
- **Infra:** Docker, Google Kubernetes Engine, Gateway API
- **CI/CD:** GitHub Actions, ArgoCD(GitOps), Cloud Functions
- **Monitoring:** Sentry, ArgoCD Notifications, Slack

1. 스펙 주도 개발(SDD)로 AI 협업 개발의 일관성 확보
AI가 기능마다 다른 방식으로 구현하는 문제 → 명세 → 계획 → 작업 순으로 개발하고 요구사항 번호(FR-xxx)를 주석에 남겨 구현 이유를 추적 → 작업 122개 중 120개 완료, 3일 만에 출시

2. TDD 기반 3단계 테스트 구축 (약 170개)
실패하는 테스트를 먼저 작성한 뒤 구현 → 단위(Vitest)·API 계약·E2E(Playwright)로 나눠 예외 흐름까지 검증 → AI를 인터페이스로 주입하고 테스트에서는 가짜 LLM으로 교체해 비결정적 응답과 무관하게 흐름 검증

3. 사주 풀이 캐싱과 개인정보 보호 설계
같은 사주도 풀이가 매번 달라지고 약 20초의 대기와 비용이 발생 → 풀이를 저장해 재사용하는 캐싱 구조 설계 → 결과 링크는 무작위 UUID, 캐시 키는 HMAC-SHA256으로 분리해 링크로 생년월일을 역산할 수 없도록 차단

4. Gateway API 기반 카나리 배포와 Slack 승인·롤백
GitHub Actions·ArgoCD로 GKE 배포 자동화 → 새 버전이 곧바로 트래픽 100%를 받는 문제를 HTTPRoute 가중치와 X-Canary 헤더로 사전 검증하는 방식으로 해결 → Slack 버튼으로 전환·롤백하는 흐름을 직접 구축해 20개 버전을 운영 반영 전에 검증

5. Sentry로 배포 버전 단위 에러 추적
커밋 SHA를 release로 주입하고 소스맵을 업로드해 에러가 난 배포와 코드 위치를 바로 확인 → 생년월일·사진·LLM 입출력은 수집에서 제외해 에러 정보만 전송

사이드 프로젝트

디자인 시스템, Storybook

관련 링크

디자인 시스템 패키지 구축 및 배포 자동화

- pnpm 모노레포로 디자인 시스템 패키지 분리, 색상/타이포그래피/spacing/radius/sizing을 CSS Custom Properties **토큰으로 설계**, **컴포넌트 구현** 및 WCAG AA 대비 기준(4.5:1)에 맞춰 색상 토큰 조정.
- 패키지 버전 관리는 **Changesets 기반**으로 구성해, 변경 사항마다 changeset을 **기록하고 버전·체인지로그를 일괄 반영**하는 구조로 관리. 빌드 파이프라인은 Turborepo로 구성해 패키지 간 빌드 순서와 캐싱을 관리하도록 설계.
- GitHub Actions로 **Storybook 빌드를 GitHub Pages에 자동 배포**하는 CI 파이프라인 구축(**디자인 시스템 패키지 변경 시에만**

	트리거).
Click-logger npm 패키지 관련 링크	이벤트 버블링 기반 **클릭 로깅 라이브러리 구현**, **Vitest 케이스로 정상/예외 케이스 검증**, **Changesets로 버전·체인지로그 관리**, GitHub Actions로 main 병합 시 npm **자동 배포까지 이어지는 파이프라인 구축**
Lint/formater npm 패키지 모노레포 관련 링크	Pnpm + Turborepo 모노레포로 oxlint/ESLint/oxfmt/Biome/Prettier 설정을 5개 독립 패키지로 분리, 포맷터 **3종(oxfmt/Biome/Prettier)** 스타일 일치를 실제 CLI출력 diff로 검증, Changesets로 버전·체인지로그 관리, GitHub Actions로 main 병합 시 **npm 자동 배포까지 이어지는 파이프라인 구축** - 신규 프로젝트마다 린트/포맷 설정을 새로 작성할 필요 없이 **패키지 설치만으로 팀 컨벤션 즉시 적용, 설정 유지보수 비용 절감** - 포맷터 3종의 출력을 동일하게 맞춰 프로젝트별로 포맷터를 **자유롭게 교체해**도 팀 컨벤션이 깨지지 않음

AI 활용 능력

대안 탐색, 프로토타이핑	여러 옵션을 빠르게 만들거나 비교해서 방향을 검증하는 역할 - Gemini의 Canvas 모드로 Mock UI를 신속 제작해 방향성 검증 및 피드백 사이클 단축 - **E2E 테스트 전략** - 3안(모킹/실백엔드 + 테스트 로그인/시나리오 문서화)을 **AI와 비교 검토** 후 채택
자동화 파이프라인 구축 및 트레이드 오프 판단	1. **CI/품질 게이트 자동화: lefthook pre-commit 훅 도입, GitHub Actions CI에 ESLint, Vitest 단위테스트 단계 통합** 2. **AI 개발 워크플로 자동화 도구 설계 및 배포** 관련 링크 - **Claude Code 플러그인 마켓플레이스를 직접 구축**해 커밋/PR/의존성 취약점 진단/ 접근성 린트 등 **4개 스킬을 개발, GitHub에 공개 배포** - 요구사항을 AI와 단계적으로 구체화**(설계안 제시 -> 승인 -> 스펙 문서화) 하는 프로세스를 정립**해, **프롬프트 즉흥 실행이 아닌 검증 가능한 설계 기반 AI 자동화** 구현 - PR/이슈 생성 등 되돌리기 어려운 작업에는 **"초안 확인 -> 승인 -> 실행" 안전 패턴을 일관 적용**해, **자동화 편의성과 통제 가능성을 함께 확보**

대외활동 / 수상

웹 서비스 개발 및 기획 동아리(멋쟁이 사자처럼 11기) 2023.01 - 2023.12	- 매주 FE 기초 개념 및 실습 중심 정기 세션 진행 - 동아리 홍보·SNS 운영, 신규 부원 온보딩, 최종일 발표 진행 등 운영 전반 담당 - 전국 62개 대학이 참여하는 동아리 연합 해커톤에서 디자인·프론트 엔드 담당으로 참여
프론트엔드 부트캠프 (kakao x goorm DEEPDIVE 프론트엔드 과정 1회차) 2025.08	- 매주 JavaScript 딥다이브·알고리즘 정기 스터디 진행 (React, TypeScript, Next.js) - 팀 프로젝트 2회: 마패(MaskPass) + Ezz(실시간 스트리밍 서비스) - 채팅 시스템 구현(호스트/시청자 구분, 닉네임 색상), Supabase Realtime Broadcast로 방(room)별 채널을 구독해 메시지 저장 없이 실시간 송수신 구현, 메인페이지·SkeletonUI 구현 - 스크럼·코드리뷰·PR 기반 협업 프로세스, 다직군 협업 기반 UT 경험
2024.12	구름톤 유니브 UGG톤 대상

학력

2021.03 - 2026.08	세종대학교 소프트웨어학과 (수료) 학점 4.0/4.5

어학 / 자격증

영어	TOEIC Speaking IH (2026.07)
2025.06	**정보처리기사** 한국산업인력공단

위의 모든 기재사항은 사실과 다름없음을 확인합니다.

작성자 : 안리안